Proyecto 1 — Informe técnico

PROYECTO N.º 1: BÚSQUEDA APLICADA A NONOGRAMAS

Asignatura: Inteligencia Artificial
Estudiante: Leonardo Araya Labarca
RUT: 21.290.314-0
Carrera/cohorte: IECI 2023
Fecha: 14 de octubre de 2026

1. FORMULACIÓN DEL PROBLEMA

Sea un tablero de altura H y ancho W. Cada celda pertenece a {vacía, pintada}. Las pistas de cada línea describen, en orden, los largos de sus bloques pintados máximos.

- Estado inicial: secuencia vacía de filas asignadas, (). Las pistas y dominios de patrones son constantes del problema.
- Estado: tupla inmutable con las primeras d filas completas, donde $0 \leq d \leq H$. Cada fila asignada satisface su pista y cada prefijo de columna es compatible con al menos un patrón completo de su pista.
- Espacio de estados: todas las secuencias parciales de patrones de fila que conservan compatibilidad con las columnas. Es un subconjunto del producto cartesiano de los dominios de filas.
- Operador: seleccionar un patrón válido para la siguiente fila y anexarlo al estado. Se descarta inmediatamente si alguna columna queda sin continuaciones.
- Prueba de meta: d = H. Como la compatibilidad con las columnas se mantuvo en cada transición, todas las pistas quedan satisfechas.
- Costo de ruta: cada asignación de fila cuesta 1; por ello g(n)=d. Toda solución tiene costo H. El interés práctico no es minimizar la longitud de la solución, sino reducir estados explorados.

2. GENERACIÓN Y PODA

Para una línea de largo L y pistas (a1,…,ak), se generan únicamente ubicaciones que incluyen al menos una celda vacía entre bloques. La línea es factible si $\Sigma ai + k - 1 \leq L$. Los patrones se memorizan con lru_cache.

Al añadir una fila, cada prefijo de columna se compara con los patrones completos válidos de esa columna. Si el conjunto compatible queda vacío, el sucesor se poda. Esta comprobación garantiza que no se acepten prefijos imposibles.

3. HEURÍSTICAS PROPUESTAS

3.1 Incertidumbre de dominios

$h_u(n) = (H-d) + (1/W)\ \Sigma_j\ (|D_j(n)|-1)$

$D_j(n)$ es el conjunto de patrones de la columna j compatibles con el prefijo actual. El primer término considera filas pendientes y el segundo favorece estados cuyos dominios de columnas ya son pequeños.

3.2 Celdas aún no forzadas

$h_c(n) = U(n)/W$

U(n) cuenta posiciones futuras para las que los patrones compatibles de su columna contienen tanto 0 como 1. Una posición es forzada cuando todos esos patrones coinciden. Menor valor significa menor ambigüedad.

Estas funciones son señales de ordenamiento ad hoc. No se afirma admisibilidad para obtener una ruta más corta, porque todas las metas están a profundidad H. La completitud proviene del espacio finito, la generación exhaustiva de patrones y la conservación de todos los sucesores compatibles.

4. ALTERNATIVAS CONSIDERADAS

BFS

Es completa en un árbol finito y visita por profundidad. Sin embargo, conserva simultáneamente muchos estados del mismo nivel. El factor de ramificación de una fila puede ser combinatorio, por lo que la memoria es su principal desventaja.

DFS

Usa poca memoria y puede encontrar pronto una solución si el orden de patrones es favorable. No utiliza la información de restricciones para ordenar alternativas; un orden desfavorable puede recorrer una rama extensa antes de retroceder. En espacios infinitos no es completa, aunque aquí el árbol tiene profundidad H.

A*

La cola de prioridad combina $f(n)=g(n)+h(n)$. Se seleccionó porque permite incorporar la información de dominios y mantiene una frontera explícita. En este problema el costo uniforme no distingue soluciones —todas tienen H acciones—, de modo que su ventaja práctica depende de la heurística y la poda. La implementación también deja Greedy para observar el efecto de usar solo h.

Decisión: se descartan BFS por consumo de memoria y DFS como estrategia principal por su dependencia del orden. Se conserva A* como diseño extensible y dirigido por restricciones. Los resultados pequeños no prueban superioridad universal; DFS puede ser más rápido en una instancia favorable.

Proyecto 1 — Informe técnico

5. CORRECTITUD

• Cada patrón generado satisface exactamente la pista de su fila.
• Un sucesor solo entra a la frontera si todo prefijo de columna coincide con al menos un patrón completo válido.
• Al alcanzar H filas, los prefijos son columnas completas; por tanto, coinciden con patrones válidos.
• Si existe una solución, sus filas aparecen en los dominios generados y ninguno de sus prefijos es podado. La búsqueda finita podrá alcanzarla.

6. COMPLEJIDAD

Para una línea de largo L con k bloques y S = Σai, la cantidad de ubicaciones posibles es, en el peor caso, C(L−S+1, k). Sea R_i el número de patrones de la fila i y C_j el de la columna j.

Sin poda, la cantidad de estados del árbol es O(Π_i R_i). Al comprobar un sucesor de profundidad d contra dominios de columnas, la versión implementada puede comparar hasta O(W · Cmax · d) valores. La cota temporal global es O((Π_i R_i) · W · Cmax · H). Es una cota deliberadamente conservadora.

La frontera de BFS/A* puede ocupar O(Π_i R_i · H · W) bits si se cuentan los tableros almacenados. DFS mantiene aproximadamente O(H · Rmax · H · W) en el peor nivel de la implementación. Los patrones precalculados requieren O(Σ_i R_iW + Σ_j C_jH).

7. EVALUACIÓN EMPÍRICA

Equipo y carga del sistema influyen en tiempos y memoria. Cada entrega debe repetir las mediciones en el computador usado para la defensa. La tabla incluida es una ejecución de referencia.

Tablero	Algoritmo	Tiempo (s)	Memoria pico (bytes)	Expandidos	Generados	Frontera máx.
5x5	A*	0.001263	14 992	7	13	8
5x5	BFS	0.016849	181 912	100	146	60
5x5	DFS	0.001536	12 832	6	12	8
10x10	A*	0.014872	193 912	73	72	7
10x10	BFS	0.013828	194 016	73	72	21
10x10	DFS	0.008457	109 672	42	46	7
15x15	A*	0.009062	33 640	16	15	1
15x15	BFS	0.010106	33 592	16	15	1
15x15	DFS	0.008682	33 592	16	15	1

El contenido de las pistas afecta más que la dimensión por sí sola. El 15x15 de franjas tiene dominios muy restringidos y resulta fácil, mientras que el 5x5 posee alternativas intermedias. Por ello no corresponde concluir que aumentar N siempre aumenta el tiempo en estas tres muestras.

8. PRUEBAS

Las pruebas verifican generación de patrones, rechazo de pistas inconsistentes y resolución de las tres instancias. Se ejecutan con PYTHONPATH=. pytest -q.

9. LIMITACIONES Y MEJORAS

La comprobación vuelve a filtrar dominios completos en cada nodo. Podría almacenarse el dominio compatible dentro del estado, aplicar propagación de restricciones, seleccionar la fila/columna de dominio mínimo y representar patrones como bits. También conviene usar varias instancias por tamaño y varias repeticiones para obtener media y desviación estándar.

10. ANEXO: REGISTRO DE APOYO CON IA

- Herramienta: asistente de Arena.ai, Agent Mode.
- Fecha: 14 de octubre de 2026.
- Consulta: solicitud de apoyo para estructurar, implementar, probar y preparar la defensa de un proyecto de búsqueda para nonogramas en Python, según el enunciado entregado.
- Decisión: se adoptó una representación por asignación de filas, se revisaron las funciones de generación y poda, se conservaron pruebas reproducibles y se documentaron límites de las conclusiones. El estudiante debe revisar, ejecutar y poder modificar cada parte antes de entregar.

Este anexo debe ajustarse para reflejar con exactitud las consultas y decisiones finales realizadas por el estudiante.